SYMBOL / CATAPULT

IntentSettlement

条件署名による原子的交換

設計書・共通仕様・Codex実装指示書

実装前の提案仕様

メイカーの条件署名とソルバーの外側署名により、2本の資産移転と注文消費を一つの取引として検証する限定仕様。

基準	内容
対象ソース	symbol/symbol@6c04695555230c89a6ffa2ac4d41774dcac77ae3
用途	同一プラグインを読み込む3ノードの隔離private-net PoC
版	0.1 / 2026-10-25
収録	対象設計書、共通仕様、Codex共通指示、対象Codex指示を全文収録
正本	同梱Markdown。署名fixtureと生成スクリプトはキット内の独立ファイル。

このPDFは実装済みコード、実行済み試験結果、安全性監査の証明ではありません。未実行の項目はNOT RUNとして扱います。図は設計本文の関係を可視化したものです。

目次

IntentSettlement PoC 設計 3

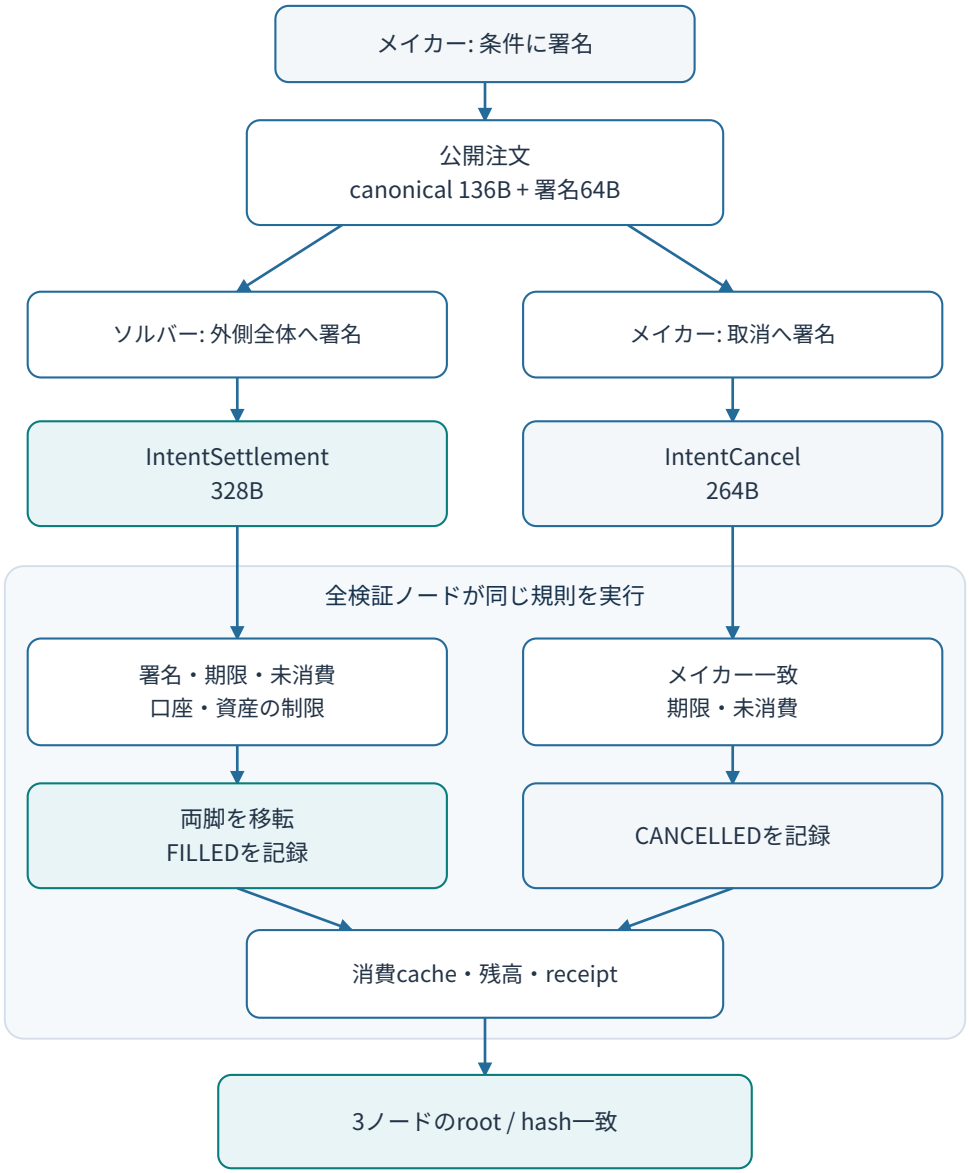
1. 目的と採用条件 . . . 3
2. 認可と不変条件 . . . 4
3. 正規化する注文binary . . . 5
4. 最上位トランザクションの形式 . . . 6
5. 検証と状態更新の設計 . . . 6
6. Consumption cacheと状態root . . . 8
7. 失敗、取消競争、分岐 . . . 9
8. ツールと照会APIの候補 . . . 9
9. 実装工程と通過条件 . . . 10
10. テスト計画 . . . 11
11. 参照した公式ソース . . . 12

両PoCの共通仕様と検証条件 13

Codex 共通作業指示 16

Codex実装指示書: IntentSettlement PoC 18

IntentSettlement PoC 設計



図版: 設計本文に記載した処理と認可の関係。実装・実行結果を示すものではありません。

版: 0.1 / 作成日: 2026-10-25 / 状態: 実装前の提案仕様

対象ソース: symbol/symbol の 6c04695555230c89a6ffa2ac4d41774dcac77ae3。本書中の IntentSettlement、IntentCancel、専用通知・cache・API名はすべて提案名であり、現在のSymbolに存在する機能ではない。数値の取引type ID、facility ID、cache IDはまだ割り当てない。

図の編集用: Mermaidソース / SVG。細かい成立条件とバイト形式は以下の本文を正本とする。

1. 目的と採用条件

メイカーが「指定した資産を指定量渡し、別の資産を指定量受け取る」という条件へ署名し、後から現れたソルバーが自己資金を使って、その条件を1つの通常トランザクションとして決済する。メイカーは注文作成後に再署名する必要がない。公開秘密鍵の中継口座P、在庫からPへの独立したfunding取引、Pからの回収取引は使わない。

外側の署名者はソルバーであり、外側の署名が資産移転を含む完成済み決済全体を認可する。メイカーの条件署名は、指定資産・数量・期限・受取口座だけを認可する。成立した決済を分割して、ソルバーの支払いだけを通常Transferとして送信できる形式にはしない。

これは合意ルールを追加するC++ CatapultプラグインのPoCである。隔離したprivate-netの全検証ノードとgenesis生成環境が同じ実装を使う。既存mainnet/testnetのノードへ一部導入して使うものではない。mainnetへのactivation、既存状態のmigration、旧ノードとの共存は対象外とする。

v1の固定範囲

項目	v1の決定
当事者	メイカー1口座、ソルバー1口座。異なる口座とする
口座認可	通常の単一署名口座のみ。実行時にmultisig化されていれば拒否
資産	異なる2つの実在するliteral mosaic ID。namespace aliasは禁止
数量	正の整数を最小単位で指定。両脚とも指定量ちょうど
約定	全量を1回だけ。部分約定・複数注文マッチングは対象外
メイカー受取先	canonical order内のrecipientをメイカー自身のaddressに固定
ソルバー受取先	外側SignerPublicKeyから導出するソルバーaddress
手数料	Settlementの通常feeはソルバーが1回負担。Cancelの通常feeはメイカー負担。独立した取引報酬はなし
注文の配布	CLIで生成した公開注文ファイルを交換。オンチェーン掲示板は必須にしない
再実行防止	専用consumption cacheでorderIdをFILLEDまたはCANCELLEDへ遷移
取消	メイカーが署名する新規最上位取引IntentCancel
内包	Settlement、CancelともAggregateへのembeddingを禁止

PoCの完成条件は、説明用シミュレーターの動作ではない。実物のC++プラグイン、取引生成・署名ツール、genesisから構築した3ノードネットワークと、状態・receipt・rollbackの統合試験結果を必要とする。

2. 認可と不変条件

正規チェーン上で決済が成立したとき、取引feeを別計上した資産移動は次の2本だけである。

1. メイカー → ソルバー: giveMosaicId を giveAmount。
2. ソルバー → メイカー: getMosaicId を getAmount。

給付資産または受取資産がネットワーク通貨でも、注文の数量はそのまま扱う。ソルバーの最終通貨残高には通常feeが追加で反映される。メイカーから注文fee、tip、暗黙のスプレッドを差し引かない。

守るべき不変条件は以下とする。

- メイカーの署名したcanonical orderを1byteでも変えると署名検証が失敗する。
- 資産の一方だけが確定した状態、または移転なしでFILLEDだけが確定した状態は存在しない。
- 同一orderIdは、同じブロック内・別ブロック・別ソルバーによる決済を含め1回しか消費できない。
- 取消が先に採用された注文は決済できず、決済が先に採用された注文は取消できない。

- 通常Transferと同じ口座・モザイク・残高制限を迂回しない。
- 一方の資産移転、consumption record、関連receiptは、ブロックのrollback時にすべて元へ戻る。
- 注文の公開データだけでは、ソルバーの単独送金署名を取り出せない。
- 同じgenesis、設定、ブロック列を与えた全ノードの状態rootとreceipt hashが一致する。

これらは価格の妥当性、流動性、検閲耐性、将来の発行者による資産操作まで保証するものではない。PoCが検証するのは署名条件に一致した一括決済である。

3. 正規化する注文binary

文字列JSONを署名対象にしない。JSONは表示・入出力用とし、署名・ID計算は次の固定136byteだけを使う。整数は符号なしlittle endian、公開鍵・address・nonceは表記文字列ではなくraw bytesである。paddingは明示したもの以外に追加しない。

offset	bytes	フィールド	規則
0	1	version	1
1	1	network	対象private-netのnetwork identifier
2	2	flags	0
4	4	reserved	全byte 0
8	32	makerPublicKey	Symbol標準の公開鍵
40	24	recipientAddress	makerPublicKeyとnetworkから導出したaddressと完全一致
64	8	giveMosaicId	literal ID、getMosaicIdと異なる
72	8	giveAmount	1..u64::MAXのうちネットワーク状態上有効な値
80	8	getMosaicId	literal ID
88	8	getAmount	1..u64::MAXのうちネットワーク状態上有効な値
96	8	validUntil	network epochからのミリ秒
104	32	nonce	注文識別用の32byte。CLIはCSPRNGで生成
計	136	canonicalOrder	余剰byte・不足byteを拒否

nonceは秘密情報ではない。合意規則として全ゼロを特別扱いしないが、通常の注文生成では新しいnonceを必ず生成する。同じ内容でnonceも同じなら同じ注文となる。整数のparseで丸め、負数のwrap、浮動小数点化を行わない。JSON表示では64bit整数を10進文字列、ID等は規定長のhexとして扱う。

条件署名とorderId

```
domainHash = SHA3_256(UTF8("SYMBOL_INTENT_ORDER_V1"))
message    = domainHash[32] || networkGenerationHashSeed[32] || canonicalOrder[136]
orderId    = SHA3_256(message)
signature  = SymbolEd25519.Sign(makerPrivateKey, message)
```

messageは200byte。domain文字列にNUL、改行、BOMを付けない。hashはKeccak-256ではなくSHA3-256。networkGenerationHashSeedは接続先から無条件に採用せず、対象private-net設定に固定した値を使う。異なるgenesisの

同network ID間でも再利用を拒否する。

署名はorderId32byteではなく、message200byteそのものに対して行う。orderIdに署名64byteを含めない。これにより同じ注文に対する署名表現や外側txの違いで、再実行防止のIDが変わらない。

Catapultの既存crypto::Sign / crypto::Verifyを参照し、標準実装と互換なEd25519を使う。既存SignatureNotification経由で検証する場合は、200byteにseedが既に含まれるためReplayProtectionMode::Disabledを指定する。Enabledではseedが自動的に付加されるので、二重prependになる。直接crypto::Verifyへ200byteを渡す実装でもよい。どちらもgolden fixtureの同一署名が通ることを必須にする。

この署名は既存Symbol通常トランザクションの署名ではなく、この新規プラグインだけが解釈する専用認可である。既存通常Transferとしては利用できない。

4. 最上位トランザクションの形式

両型とも標準の128byte Transaction headerを使う。headerのversionは1、networkはcanonical orderと一致、外側の署名とハッシュは標準経路で計算する。dataBufferが本文全体を署名対象に含むことをテストする。追加の外部cosignatureや変更可能な署名対象外footerは設けない。

型	外側署名者	payload	合計size
IntentSettlement	ソルバー	canonicalOrder136B + makerSignature64B	328B
IntentCancel	メイカー	canonicalOrder136B	264B

Settlementのoffset128からorder、264からmakerSignature。Cancelのoffset128からorderを置く。orderIdは両方とも再計算し、冗長なorderIdフィールドを送らない。取引typeの数値はM0で既存登録と衝突しない私設用割当を調べ、schema・C++・ツールで共有する。supportsTopLevel=true、supportsEmbedding=false、embeddedCount=0とする。内部の2本は既存のembedded Transactionではなく、この型の固定セマンティクスなのでDeepCount上は最上位取引1件として扱う。

Cancelではouter.SignerPublicKey == order.makerPublicKeyが必須。取消自体を外側署名が認可するため、別のmakerSignatureは不要。完全なcanonical orderを含めることで、orderIdとメイカーを他人が偽って紐付ける問題を避ける。任意のorderIdだけを渡して他者注文をCANCELLEDにできるAPIにしない。

2種類の期限

外側Deadlineは通常取引の寿命制限を受ける。注文のvalidUntilは別の値で、外側Deadlineが過ぎた場合にソルバーは新しい外側取引を作れるが、同じ注文を期限後に復活させることはできない。

PoCでは、SettlementとCancelの実行時に次を要求する。

```
BlockTime < validUntil
validUntil - BlockTime <= 21_600_000  // 6時間、減算前に上下関係を検査
```

等号BlockTime == validUntilで注文は期限切れ。合意判定に実行マシンの壁時計を使わない。canonical orderに作成時刻はないため、この上限は「署名から6時間」ではない。遠い将来のvalidUntilを署名した注文は、当初はExpiryTooFarで受理されず、期限の6時間前から受理可能になる。この意味をCLIと試験に明記する。より厳密な発行時刻を将来求めるなら、別versionでissuedAtを追加する。

5. 検証と状態更新の設計

5.1 Stateless検証

- payloadの厳密size、固定version、flags/reserved、networkを検査する。

- 公開鍵からrecipientを導出して一致を検査する。
- 2つのliteral mosaic ID、正の数量、異なる資産という構造を検査する。
- Settlementでは専用domain、設定されたgenesis seed、canonical orderで200byteを再構築し、maker署名を検証する。Cancelには別のmaker署名フィールドがなく、次項の外側署名で認可する。
- Cancelは外側署名者とmakerの一致を検査する。SettlementはmakerとsolverのAddressを検査する。
- 外側署名は既存標準の署名経路でも必ず検査する。

5.2 Stateful検証

実行するブロックの時点で、期限、未消費、口座状態、mosaic状態を再確認する。RESTで先に検査した結果を合意判定の代わりに使わない。

multisig判定は、口座が他者のcosignerとして登録されているだけの状態と、その口座自身がmultisigになっている状態を区別する。v1で拒否するのはその口座の送金権限がmultisig化されている場合である。署名後にmakerがmultisigへ変わった場合も拒否する。

注意: 既存のOperationRestrictionとMultisigPermittedOperationはTransactionNotificationを監視する。外側の通知はsolverにしか適用されない。BalanceTransferを2本発行するだけでは、makerの取引型制限が抜ける。専用validatorで両者のmultisig状態と操作制限を検査する。

v1の明示的な操作制限規則は、makerとsolver双方について、outgoingのTransfer型とIntentSettlement型の両方が許可されることとする。solverの外側IntentSettlement型は標準経路でも検査される。Cancelは通常の外側IntentCancel型の制限に従い、資産移転の制限は検査しない。makerの取消に残高や対象mosaicの有効性を要求しない。ただし通常feeを払えることは必要である。

住所相互制限、受取側mosaic制限、mosaicのtransferability・存続期間・global/address restrictions、残高を、既存Transfer相当の通知で検査する。Balance observerへ直接書き込んで既存validatorを迂回する実装は禁止する。操作制限の検査を流用するためだけに偽のTransactionNotificationを追加してはならない。同じ通知に結び付くHashCacheや他のobserverも発火するためである。

5.3 初期残高とfee

PoCの単純化として、makerはこの取引の適用前にgiveAmountを保有し、solverはgetAmountとouter.MaxFeeの全額を適用前に賄えることを要求する。口座別・mosaic ID別の必要額mapを作り、getMosaicがネットワーク通貨なら両額をoverflow検査付きで合算する。受取予定の資産を先に使って不足を補うことはv1では認めない。これは既存Symbolの一般的な要件ではなく、このPoC独自の保守的な制約である。

この「適用前残高」を専用validatorで正しく確認するため、共通仕様に従いprivate-netのTotalVotingBalanceCalculationFixをHeight(1)へ設定し、通常ブロックでcustom通知→fee debitの経路を使う。設定ファイル上の正確なキー・値の表記と受理はM0で確認する。fork heightが未来の古い設定を流用すると、custom前にfeeを引いた残高へ再びMaxFeeを要求してしまうので禁止する。M0の小試験で実際の通知順をassertし、通常ブロックの先頭から対象設定が有効である証跡を残す。

実際の引き落としは標準処理が計算するouter.Size × block.FeeMultiplierだけ。MaxFee全額を新プラグインが引き落としたり、内部脚に別feeを請求したりしない。fee倍率0だけで成功しても受入完了としない。

5.4 通知の順序

以下の新規通知名は設計上の候補であり、既存APIではない。実装時は現行通知channel、observer undoの方式、factoryを読んで適切な型へ落とす。

1. IntentAuthorizationNotification（提案、validator channel）: canonical order、maker署名、solver、外側hash/maxFeeを渡し、構造・署名・期限・未消費・初期残高・操作制限を検査。

2. 実在するAccountPublicKeyNotification / AccountAddressNotification: 両者を正しく登録。maker公開鍵通知は専用認可の検査を通った後に置く。
3. 実在するAddressInteractionNotification: maker→solver、solver→makerの両方向についてTransfer相当として発行。
4. 実在するBalanceTransferNotification: maker→solverのgive、solver→makerのgetをこの順で発行。
5. IntentConsumedNotification（提案、validator＋observer channel）: orderId、FILLED、makerAddress、validUntil、height、外側hashを保存。挿入直前にも未消費を検査。
6. 外側標準経路: 通常のfee・署名通知を通常のfork設定に従って処理。

個別のBalanceTransfer通知では、その脚の制限検証→状態変更が順次行われる。2本目や最後のfee処理で失敗した場合でも、1本目だけが確定してはならない。採掘候補適用でのundoと受信ブロック拒否の両方を試験する。

Cancelは専用取消認可の検証後にCANCELLEDを保存するだけで、資産移転を発行しない。外側の通常手数料は署名者のメイカーが払う。

6. Consumption cacheと状態root

注文の全情報を合意状態に登録する必要はない。未消費の注文は公開署名データとして配布し、状態には終端になった注文だけを保存する。

要素	内容
key	orderId 32byte
status	1=FILLED、2=CANCELLED。他値は不正
makerAddress	24byte
validUntil	u64
consumedHeight	u64
outerTransactionHash	32byte

cache valueのcanonical serializerは、status1B、reserved7B、makerAddress24B、validUntil8B、consumedHeight8B、outerTransactionHash32Bの80Bとする。整数はLE、reservedは0。メモリ構造体の暗黙paddingをhashしない。Patricia/Merkle対応のsub-cacheを登録し、格納順序に依存せず同じrootを得る。

既存cacheにない注文はOPEN。照会時にBlockTime >= validUntilならEXPIREDと導出する。保存済みFILLED/CANCELLEDは期限後もその状態を返す。OPENは「現在必ず実行可能」という意味ではなく、残高・制限・ExpiryTooFarなどの可否理由を別フィールドで返す。

PoCでは終端レコードをpruneしない。期限切れ直後の削除は深いrollbackや再検証と競合し得るため、履歴保持とpruningは別設計にする。

新しいMerkle対応cacheを追加すると、stateHashを構成するcache root群にも影響する。 nemgenと3ノードに同じプラグイン・cache登録・設定を入れ、対応するgenesisから生成し直す。既存mainnetの9本rootという前提をそのまま使わない。baselineと導入後のroot構成・cache ID・空cache root・genesis hashをテスト成果物に記録する。機能を入れたノードだけ別stateHashを出す状態を成功とみなさない。

commit、undo、reorg

commitではOPENから終端レコードを1件insertする。rollbackでは、その取引が作成したレコードと移転を逆順に戻し、注文を元の未登録状態にする。別の取引のレコードを誤って削除しないよう、orderId・status・height・outerHashの一致を不変条件として検査する。

同一ブロックに2回の消費があれば、1件目のdeltaが2件目の検証に見える必要がある。プロセス終了前だけ記憶するset、ソルバーのローカルDB、REST上の既約定検索で代用しない。

7. 失敗、取消競争、分岐

状況	必須の挙動
maker署名不正	Settlementを拒否。資産・consumptionに変化なし
第2脚だけ残高／制限不正	Settlement全体を拒否。第1脚も残らない
settle→settle	先行のみ有効、後行はAlreadyFilled
cancel→settle	先行取消のみ有効、後行はAlreadyCancelled
settle→cancel	先行約定のみ有効、後行はAlreadyFilled
cancel→cancel	先行のみ有効。別外側hashでも再取消は拒否
候補作成時の失敗	外側1件全体をundoし、その候補を外せる
無効な外側を含む受信ブロック	ブロック全体を拒否。チャンクだけ捨てて残りを採用しない
未finalizedの決済ブロックをrollback	資産、consumption、receiptが先行状態へ戻る
rollback後の別ソルバーによる再約定	期限と他条件を満たせば可能。二重の確定残高は生じない

優先順位はブロック内で確定する順序だけで決まり、ローカルで先に取消を送信したという事実だけでは保証しない。CLIはbroadcast、unconfirmed、confirmed、finalizedを区別し、取消送信直後に取消済みと表示しない。

8. ツールと照会APIの候補

実装はまずCLIで署名からアナウンスまでを再現できるようにする。名称は提案で、対象repoの実在するbuild・SDK・REST構造に合わせる。

- intent-order create: 対象ネットワーク、2資産、整数数量、validUntilからcanonical bytesとmaker署名を生成。秘密鍵は出力しない。
- intent-order inspect/verify: canonical bytes、200byte message、orderId、表示数量、検証結果を出す。公開データだけで動く。
- intent-settle build/announce: 同じ注文を受け取り、solverが外側を署名。未確認のseedやnetworkに送らない。
- intent-cancel build/announce: makerがcanonical orderを含むCancelに署名。
- intent-status: 固定height/hashと注文データを指定して、合意cache状態と実行可能性を照会。

候補RESTはGET /experimental/intents/{orderId}とPOST /experimental/intents/status。前者は保存済みレコードまたはUNKNOWNを返す。未知のorderIdだけではvalidUntilが分からないため、OPEN/EXPIREDを断定しない。 後者はcanonical orderと署名を受け、指定または応答中で明示したブロック時点でOPEN/EXPIRED等を導出する。POSTだが状態を書き換えない照会とする。

RESTは任意の便利機能であり、合意処理は各ノードが再検証する。既存RESTにこれらのルートがあると仮定しない。実際に実装した場合はroute登録、schema、cache読取、height整合性までテストする。未実装ならCLIとintegrationの直接cache照会を使い、文書に未実装と記す。

9. 実装工程と通過条件

段階	作業	通過条件
M0	repoのAGENTS、build/test、schema生成、nemgen、plugin登録、cache/serializerの既存例を実読	baseline一致、私設ID割当、build手順、実在APIとの対応表を記録
M1	canonical serializer、署名、ID、両型のschemaとC++構造	固定sizeとcross-language golden fixture一致
M2	新TransactionPlugin、stateless/stateful validators、既存transfer通知への接続	無効署名・制限・残高・feeを含む実C++ unit tests
M3	consumption cache、commit/undo、永続化、root計算	重複・取消・再起動・異なる挿入順の検証
M4	CLI、3ノードprivate-net、専用genesis	非ゼロfeeの正常約定／取消を全ノードが同じhashで受理
M5	競争、失敗候補undo、受信無効block、fork/reorg	資産・状態root・receipt・consumptionが仕様どおり
M6	再現手順・raw証跡・既知限界	別環境で最初から再実行できる

M0で型を追加するだけでは正しいobserverやcacheへ接続できないと判明した場合は、必要な最小のコア変更を理由とともに分離する。プラグインだけで可能だと証明するために検査を省かない。

10. テスト計画

ID	試験	期待結果
IS-01	serializerの全offset、136/200/328/264B	完全一致、余剰byte拒否
IS-02	fixtureをC++と独立ツールで署名・検証	同じmessage/orderId、標準C++で署名検証成功
IS-03	field改変、wrong seed/network、domain、二重seed	すべて拒否。署名にIDだけを使う実装も不一致
IS-04	flags/reserved、0 amount、同mosaic、alias	拒否
IS-05	expiryの前1ms・同値・後1ms、6h境界	規定のstrict expiryとupper capに一致
IS-06	fee倍率非ゼロ、give/getそれぞれがXYM	maker数量は固定、solverだけ実fee1回
IS-07	maker/solver残高不足、MaxFee合算overflow	全体拒否、全状態不変
IS-08	maker/solverの実行直前multisig化	両方拒否。単なる他口座cosigner登録と区別
IS-09	両者のTransfer/Settlement操作制限	どの拒否条件も迂回不可
IS-10	住所制限、受取mosaic制限、mosaic global/address制限	通常Transferと同じ可否
IS-11	存在しない／期限切れ／非transferable mosaic	前2者は拒否。非transferableはownerが送受信者かによる例外を含め通常Transferと同じ可否
IS-12	2ソルバーが同注文を同block／別blockで実行	1回のみ。外側hash違いで再実行不可
IS-13	cancel/settleの両順、再取消、他人のCancel	包含順の先行だけ成立、他人は拒否
IS-14	1脚目適用後の後段validator失敗	C++ pipeline試験で1脚目observe後に失敗を注入し、UT undoで全資産・cache・receiptが元通り。integrationでも無効注文の前後不変を確認
IS-15	不正Settlementを含む受信block	block全体拒否、先行正当txの状態もcommitしない
IS-16	非finalized blockのrollbackと別枝で再約定	consumption削除・資産復元・再実行が一致
IS-17	FILLED/CANCELLED後に全ノード再起動	状態維持、再消費拒否、root一致
IS-18	fresh genesisと全nodeのstateHash構成	新cache含めroot構成・genesis一致
IS-19	プラグインなしの隔離ノードへ新型を渡す	受理不可を確認。既存網互換と誤認しない
IS-20	取引取り出し・並べ替え・再包装	署名済み単独fundingとして再利用不能

モデル上の試験とCatapult実行試験を別欄に記録する。各IDに実行コマンド、コミット、設定、exit code、raw log、tx/block hash、前後残高、状態root・receipt hashを紐付ける。IS-14の故意の失敗注入はtest harness限定とし、本番プラグインに検査回避用の設定を作らない。まだ実行していない項目はNOT RUNとする。状態遷移をPythonだけで模倣してPASSにしない。

11. 参照した公式ソース

以下はすべて固定baselineへのリンクである。M0ではリンク先の実ファイルと周辺testを改めて読む。

- PluginManager.h: 取引、validator、observer、cacheの拡張点。
- TransactionPlugin.h: 署名対象buffer、embedding、通知context。
- Transaction.h: 標準128byte header。
- NotificationPublisher.cpp: 外側のfee、deadline、署名、固有通知の順序。
- Signer.h: 標準Sign/Verify。
- TransferTransactionPlugin.cpp: AddressInteractionとBalanceTransfer通知。
- OperationRestrictionValidator.cpp: 外側だけではmakerに適用されない操作制限。
- MultisigPermittedOperationValidator.cpp: multisig口座自体からの通常操作禁止。
- MosaicRestrictionBalanceValidator.cpp: 送受信双方のmosaic restriction。
- HarvestingUtFacadeFactory.cpp: 候補取引のapply/undo。
- BatchEntityProcessor.cpp: block内の逐次検証と失敗。

設計図はdiagrams/intent-settlement.mmd、実装依頼文はcodex/intent-settlement-task.mdを参照。

両PoCの共通仕様と検証条件

仕様版: 0.1 / 2026-10-25 / ステータス: 実装前のPoC仕様

1. 対象と結論

IntentSettlementは、メイカーの条件署名とソルバーの決済署名を検証する交換専用取引を追加します。AtomicBundleは、限定された既存署名済みAggregateと、その前後の内部操作を一体で実行する取引を追加します。

共通する目標は、ハーベスターの善意や取引の並べ方に依存せず、支払認可と資産移動の不可分性を検証規則で強制することです。外側の箱を作るだけでは、独立に有効な通常Transferの署名を失効させられません。

2案は独立した比較対象です。IntentのコードをAtomicへ自動的に取り込まず、共通化は両方の最小仕様が安定してから別変更で行います。

2. 固定した一次資料

資料	確認対象
symbol/symbol	6c04695555230c89a6ffa2ac4d41774dcac77ae3
heikaの元の交換例	891a2cb17a0d52c85ed85594d8283c844481a6cb
公式プラグイン説明	docs.symbol.dev / plugin.html
公式Aggregate説明	docs.symbol.dev / aggregate-transaction.html

以下のAPI名は固定コミットで確認した実在の拡張点です。新しい取引型名・通知名・キャッシュ名は各設計書の提案であり、既存APIではありません。

- PluginManager: 取引型、validator、observer、cacheの登録。
- TransactionPlugin: 署名対象、追加Merkleデータ、embedded件数、top-level可否。
- AggregateTransactionPlugin: Aggregateはembedded不可。子の固有通知と共通の取引処理は別。
- NotificationPublisher: 期限、手数料、通常署名、取引ハッシュ通知の共通経路。
- HarvestingUtFacadeFactory: 候補取引への適用と失敗時undo。
- UniqueTransactionHashValidator と TransactionHashObserver: 既存取引ハッシュの重複防止とrollback。
- CatapultCache: サブキャッシュrootから全体StateHashを計算。

3. ネットワーク境界

3.1 私設ネットワークのみ

公式C++ catapultを対象とし、同一バイナリ・同一設定の独立した3プロセス以上で検証します。RESTだけのモデルやメモリ上の独自台帳は補助資料です。プラグインのロード、ブロック受け入れ、状態更新、別プロセスへの伝播を確認して初めてネットワーク試験が成立します。

公開mainnet/testnetのseedノードを接続先に使いません。鍵と資金は使い捨ての私設ネットワーク専用とし、既存ウォレットの秘密鍵を入力しません。ネットワーク識別子だけで環境を区別せず、私設チェーン固有のgeneration hash seedで署名を分離します。

3.2 新しい状態rootとgenesis

新しいMerkle対応サブキャッシュは、空であってもStateHashの構成に影響し得ます。Intentの新キャッシュを、既存チェーンのスナップショットへ単に後付けしてはいけません。nemgen側と全検証ノード側が同じプラグインとキャッシュ構成を持つ、fresh private genesisを作ります。

PoCの検証有効化高さは 2 を初期案とします。これは全ノードが初めから新プラグインを持つ私設ネットワーク内の境界試験です。既存メインネットへの安全な状態移行や、旧ノードと共存できることを実証する設定ではありません。

旧バイナリへの負試験は、新しい取引型のデコード／検証拒否を確認します。IntentではgenesisのStateHash構成も異なり得るため、旧バイナリが新チェーンの最初のブロックから追従できることを前提にしません。

外側feeの通知順を固定する

基準ソースの NotificationPublisher は、fork設定によりCustom通知より前にfeeを引く場合と、後に引く場合があります。PoCでは取引型の有効化高さ2以降をCustom処理 → 外側fee debitへ固定します。ForkHeights.TotalVotingBalanceCalculationFix の私設値を Height(1) とする構成を初期案とし、実際の設定キー表記・受理・通知順をM0で確認してください。

古いpresetの大きなfork高さをそのまま流用すると、プラグインが初期残高として「支払額 + MaxFee」を検査する前にfeeが減り、二重にfee資金を要求する不整合が生じます。最大手数料とちょうど同額のfee資金を持つ境界ケースを必須試験にして、この順序を確認します。mainnetのfork設定を変更する指示ではありません。

既存Aggregate V3の有効化も揃える

AtomicBundle v1はLegacy AggregateComplete V3を検証します。新しいAtomicBundleだけを高さ2で有効化しても、既存Aggregateのversion validatorがV3を拒否する設定では動きません。私設ネットでは ForkHeights.StrictAggregateTransactionHash と ForkHeights.UniqueAggregateTransactionHash も Height(1) を初期案として設定し、ネイティブの単独V3とBundle内V3の両経路がテスト開始時から同じ版・hash規則を使うことを確認します。キー表記・genesis生成との整合・境界の受理はM0の必須確認です。既存mainnetのfork履歴を変更する意図はありません。

3.3 型番号とABI

取引型の数値、FacilityCode、キャッシュ登録番号をこの文書で既存番号へ仮割当しません。CodexはM0で対象コミットのenum・登録処理・予約値を確認し、衝突のないPoC専用値を protocol-abi.md に記録します。クライアント、catbuffer、ノードは同じ定義を参照します。

4. 共通の安全性条件

ID	必須条件
C-01	認可されていない第三者の残高を減らさない。outer署名で認可される口座と、条件署名で認可される口座を区別する。
C-02	交換の片側、資金投入だけ、注文消費だけを確定させない。
C-03	同じ注文を同一ブロック・後続ブロック・別の包み方から二重実行できない。
C-04	reorg時は残高、注文消費、必要なreceiptとcacheを一貫して元へ戻す。
C-05	ネットワーク、取引型、版、金額、宛先、期限、手数料条件を署名対象と整合させる。
C-06	マルチシグ、アカウント制限、モザイク制限・存続期間などを迂回しない。v1で非対応の状態は明示的に拒否する。
C-07	型付き・長さ付き・順序付きの正規バイト列を使う。JSON文字列表現を署名しない。
C-08	全整数計算に上限とoverflow検査を置く。金額にfloatを使わない。
C-09	外側の実サイズに基づくネットワーク手数料を1回課金し、ブロックの手数料分配・集計と一致させる。
C-10	同じ入力・前状態から全ノードが同じ状態とreceiptを生成する。外部HTTP・壁時計・乱数を合意処理で参照しない。

5. 原子性の意味

候補ブロックを作る側では、無効な新取引の一時的変更をundoし、候補から除ける必要があります。受信側では、無効な新取引を含むブロック全体を拒否します。受信した署名済みブロックから不正な部分だけを削除して採用する設計ではありません。

署名・形式検査、状態検査、資産移動、注文消費の通知順序を明示し、途中失敗を挿入して最終状態が変化しないことを試験します。observerのCommitとRollbackだけでなく、未承認候補のundoも対象です。

6. PoCの完成条件

完成には次の証拠が必要です。

1. 対象コミットと変更差分、ビルド環境、再現用コマンドを記録する。
2. ネイティブのC++プラグインがビルド・ロードできる。
3. 独立したクライアント側codecとC++側codecの固定ベクトルが一致する。
4. 各設計書の必須テストが実行され、出力と結果が保存されている。
5. 3ノードで正常取引を受け入れ、同一高さのblock hash・StateHash・ReceiptsHashが一致する。
6. 未ファイナライズの競合枝への実際の切り替えを起こし、巻き戻し・再適用後の残高と注文状態が一致する。
7. 1ノードを再起動／再同期しても、その状態と重複防止が維持される。
8. 未実施、失敗、実装済みのみ、実行成功を区別した最終報告がある。

PoC完成は、mainnetへの採用可能性や資産保管サービスとしての安全性を保証するものではありません。公開ネットワーク導入には状態移行、費用・DoS、独立レビュー、クライアント互換性など別の設計判断が必要です。

7. 実装範囲の優先順位

最初は各案の限定仕様を満たします。任意の再帰、部分約定、価格オラクル、AMM、ブリッジ、外部資産、一般公開UIは含めません。ネットワーク処理と独立したCLIで十分です。REST/Mongo/ウォレットの本格対応は、合意処理のPoCが通った後に進めます。

C++だけに閉じたバイト列の自己検証ではクライアント相互運用を確認できません。一方、単純な参照モデルだけではコンセンサスを確認できません。両者の役割を分けて証拠を残します。

Codex 共通作業指示

この文書は、IntentSettlementとAtomicBundleのどちらの作業でも最初に読みます。現在のリポジトリの AGENTS.md、ビルド規約、ユーザーから与えられた制約を確認したうえで適用してください。

1. 依頼の完了条件

設計書に基づき、公式C++ catapultへ新取引型を実装し、対象テストと隔離した私設ネットワークによる受け入れ試験を行ってください。設計説明、疑似コード、HTTPのモック、Pythonの残高モデルだけで完了しないでください。

不足する環境や権限がある場合は、実行可能な作業を進め、未実施部分・理由・必要な手順を具体的に記録します。試験結果を捏造せず、単体試験の成功をネットワーク検証の成功と表現しないでください。

2. 開始時の確認

1. README.md、docs/00-common-spec.md、対象案の設計書と指示書を読む。
2. 実装先リポジトリを確認する。存在しなければ公式 https://github.com/symbol/symbol から隔離したローカル作業コピーを用意する。
3. gitの現在のブランチ・コミット・未コミット変更と、適用される AGENTS.md を確認する。既存作業を上書きしない。
4. 基準は 6c04695555230c89a6ffa2ac4d41774dcac77ae3。異なるコミットへ実装する場合は差分とAPI変更を先に記録し、黙って基準を変えない。
5. Intentは poc/intent-settlement-v1、Atomicは poc/atomic-bundle-v1 を候補に、別作業ツリーを使う。既存名があれば衝突しない名前を選び、記録する。
6. ビルド手順・CMake構成・ネイティブテスト配置・nemgenとプラグインの読み込み方を実ソースから確認する。コマンドや既存APIを推測で作らない。

3. 作業の進め方

- M0から順に進め、各段階で具体的なファイルと検証結果を残す。通常の実装判断ごとにユーザーへ確認を要求しない。
- リポジトリ内の検索は rg を優先する。依存のない調査や読み取りを並行し、同じファイルへの編集を競合させない。
- 設計の安全性条件を緩める代替実装は禁止する。実在APIと文書案が合わない場合は、機能を保った最小変更を行い、Architecture Decision Recordに根拠を記録する。
- マルチシグやモザイク制限に対応しきれなければ、v1の範囲に従って明示的な失敗にする。黙って検査を省かない。
- 広範なリファクタリングや、関係のない依存更新を混ぜない。
- CLI引数やログへ実用秘密鍵を露出させない。fixture鍵は公開済みの使い捨てテスト専用と明記する。
- ネットワークアクセスはソース取得・依存取得と隔離テストに限定する。公開チェーンへ取引を送信しない。

4. 実装前に固定するもの

次を docs/poc/<案名>/protocol-abi.md に記録してください。配置はリポジトリ規約に合わせて変更できますが、参照を更新します。

- 取引型、版、FacilityCode、キャッシュ番号。予約値との衝突検査結果。
- 固定・可変バイト列のレイアウト、endianness、padding、署名対象、hash、全長上限。
- 合意設定、有効化条件、genesis作成条件、既存機能から変える箇所。
- 外側feeの通知順。共通仕様に従い、PoCの全実行高さでCustom処理の後にfeeが引かれる設定と実際の順序を確認する。
- 手数料、DeepCount、receipt source、署名検証コストの扱い。
- 失敗コードと、どの検査で拒否するか。

型名や関数名は「新しく作るもの」と「既存の呼び出し先」を区別します。既存のコアAPIでない名前を、実在するかのように呼び出すコードを残さないでください。

5. 検証の方針

最初に対象コンポーネントのテストを実行し、その後に関連する既存のAggregate・Transfer・HashCache・restriction・rollbackの回帰試験を行います。対象が通っただけで終了せず、設計書の各必須受け入れ項目を根拠付きで埋めます。一方、残ったリスクや必須ゲートに関係しないテストの拡大は避けます。

私設ネットワーク試験では、全ノードの同一高さを比較します。時刻が異なる任意の先端同士を比較して一致したと扱わないでください。fork試験は、ブロックを実際に作成・伝播して巻き戻しを起こします。単にobserverのRollbackを呼ぶ単体試験は、別の検証項目です。

新Merkleキャッシュを持つIntentでは、nemgenとノードに同じプラグインを読み込ませます。既存mainnetのgenesis・snapshotへキャッシュを追加する方法は、このPoCの代替になりません。

6. 残すべき成果物

各案について、少なくとも次を残してください。

成果物	内容
実装	C++ plugin、model、notifications、validators、observers、必要なcacheと設定
クライアント	注文作成・署名、取引構築、送信、結果確認の最小CLI
スキーマ	catbuffer／正規codec、署名・hash・サイズの固定ベクトル
私設ネット構成	fresh genesis、3ノード設定、起動・停止・データ初期化・再現手順
テスト	正常系、改ざん、権限、重複、fee、undo、fork、再起動
証拠	コミット、設定hash、raw payload、期待値、実測値、block/state/receipt hash、ログ
報告	実装済み／実行済み／未実施、残る課題、変更理由、再現コマンド

証拠の既定配置は artifacts/poc/<案名>/ とします。大量ログ・秘密鍵・ビルド出力を無差別にgitへ追加しません。再現に必要な小さい公開テストfixtureとサマリーを選別します。

7. 最終報告の書式

次の順で報告してください。

1. どの機能が動き、どの必須条件がまだ満たされていないか。
2. 対象コミット、作業ブランチ、主要変更ファイル。
3. 各受け入れテストIDと PASS / FAIL / NOT_RUN、証拠へのリンク。
4. 3ノード・fork・再起動の結果。実行していなければ明確に記す。
5. 設計書からの変更と理由。
6. 同じ環境で再現するための正確なコマンド。

この依頼はローカルPoCの実装と検証です。リモートへのpush、PR公開、mainnetへの配置、実資産を使った運用開始は含みません。

Codex実装指示書: IntentSettlement PoC

この指示書を、共通資料と設計書を参照できるCodexの作業環境へ渡してください。以下は実装を依頼するための文章です。現在の成果物は設計であり、プラグインやテストが完成したことを意味しません。

依頼

Symbolの署名済み注文を1回だけ原子的に決済する、実物のC++ Catapultプラグインを実装してください。対象baselineはsymbol/symbolの6c04695555230c89a6ffa2ac4d41774dcac77ae3です。モデルコードだけで終了せず、対応genesisから起動した3ノードprivate-netで相互運用、非ゼロ手数料、取消、二重実行、rollbackを検証してください。

最初に、次の順で読んでください。

1. 作業repo内の適用対象AGENTS.md、build/test/developmentの実在する指示。
2. docs/00-common-spec.md と codex/00-common-instructions.md。
3. docs/intent-settlement-design.md。
4. fixtures/README.md、fixtures/intent-order-v1.json、fixtures/generate_reference_vectors.py。
5. 設計書の固定commitへの一次資料と、対応する既存unit/integration tests。

このキットをsource repoへコピーする場合は、相対参照を追える場所に配置してください。他PoCのAtomicBundleとは独立したbranch/worktreeまたは独立した設定で進め、競合するtype/cache割当を勝手に混在させないでください。

1. 最終成果物

- 新しい最上位IntentSettlementとIntentCancelのschema、C++モデル、TransactionPlugin、登録処理。
- 正規注文codec、maker条件署名、orderId生成、solver外側署名、取消を生成するCLI。
- stateless/stateful validatorsと既存のTransfer検証経路に従う資産移動。
- 永続化とMerkle root計算に対応したconsumption sub-cache、commit/undo。
- 基準fixtureに対するC++検証と独立ツールのcodec照合。
- 同じplugin・設定・genesisを使う3ノードの再現可能な統合環境。
- 正常・失敗・取消競争・reorg・再起動の試験証跡。
- 実際のbuild/実行コマンド、制限、未実装項目を記したREADMEと最終報告。

RESTの独自routeは必須ではありません。CLIと統合試験で必要な状態を確認できれば先に進めてください。未実装routeを既存機能のように文書化してはいけません。

2. 変更してはいけないv1の条件

メイカー1名、ソルバー1名、異なるliteral mosaic IDを2つ、正の整数数量、全量1回の決済です。maker/solverは異なる単一署名口座とし、実行時にmultisig口座になっていれば拒否します。recipientはmakerのaddressに固定します。namespace alias、部分約定、複数maker、注文報酬、公開秘密鍵P、独立したfunding/sweep、任意の通常取引の内包、Aggregateへのembeddingは実装しません。

決済時の資産移動は次の2本だけです。

```
maker -> solver: giveMosaicId / giveAmount
solver -> maker: getMosaicId  / getAmount
```

Settlementのfeeはsolverが外側取引で1回払います。Cancelの通常feeはmakerが払います。決済時のmakerのgetAmountをfeeや報酬の名目で減らさないでください。PoCでは両者に実行前の資産を要求し、solverはgetAmountに加えてouter.MaxFeeを事前に賄うものとします。口座別・mosaic別mapへ必要額を加算し、同一通貨の合算はoverflowを検査してください。このMaxFee事前残高要件はPoC独自の保守的な制約です。実際のfee引き落としは標準処理だけで行います。

共通仕様に従いprivate-netのTotalVotingBalanceCalculationFixをHeight(1)へ設定します。設定表記と受理をM0で確認し、通常ブロックではcustom通知の後にfeeが引かれることをassertしてください。feeが先に引かれる古いpresetを使い、専用validatorで再びMaxFeeを要求する二重計上は不可です。

3. binaryと署名を厳密に固定する

canonical orderは設計書のoffset表どおり136byteです。

```
version:u8=1 | network:u8 | flags:u16LE=0 | reserved:u32LE=0
makerPublicKey:32 | recipientAddress:24
giveMosaicId:u64LE | giveAmount:u64LE
getMosaicId:u64LE  | getAmount:u64LE
validUntil:u64LE | nonce:32
```

署名・識別は以下を変更せず実装してください。

```
domainHash = SHA3_256(UTF8("SYMBOL_INTENT_ORDER_V1"))
message = domainHash || networkGenerationHashSeed || canonicalOrder
orderId = SHA3_256(message)
makerSignature = SymbolEd25519.Sign(makerPrivateKey, message)
```

messageは200byteです。NUL・改行・BOM、JSON、hex文字列、orderIdだけを署名してはいけません。Keccak-256へ置き換えないでください。orderIdにmakerSignatureを含めないでください。

baselineのcrypto::Sign / crypto::Verifyを使うか、その標準互換性をfixtureで証明してください。SignatureNotificationを流用する場合、messageにseedが既にあるためReplayProtectionMode::Disabledを指定します。Enabledによるseed二重prependを必ず拒否テストへ入れてください。

Settlementは標準128B header＋order136B＋maker署名64B＝328B。Cancelは標準128B header＋order136B＝264B。外側署名はそれぞれsolver/makerで、本文全体を署名対象にします。Cancelは外側署名者とorder.makerPublicKeyの一致を要求し、orderIdを再計算します。orderId単体だけで取消を認可しないでください。

最上位のみを許可し、supportsEmbedding=false、embeddedCount=0とします。新規type/facility/cacheの数値を推測で固定せず、M0で登録済み値・既存実験規約を調べ、私設用の衝突しない値をschemaと実装で共有してください。

4. 検証と通知の注意

外側標準の署名・deadline・fee検証を保持したまま、専用認可通知でmakerの条件署名、期限、未消費、口座、初期残高を検査してください。実行可能期限は厳密に次の式です。

```
BlockTime < validUntil <= BlockTime + 21_600_000
```

加算overflowを避け、上下関係検査後の差で比較してください。validUntilと同時刻でEXPIREDです。外側deadlineの標準規則とは別です。注文に作成時刻がないため、遠い将来の期限は最初ExpiryTooFarでも、時間が進むと受理可能になります。これを「署名から6時間」と表示しないでください。

以下の制限迂回を防いでください。

- multisig判定は「その口座のcosignatoryAddressesが空でないか」に相当する既存規則を確認します。単なる他口座のcosignerであるだけのエントリを理由に拒否しません。
- makerとsolver双方でTransfer型とIntentSettlement型のoutgoing操作許可を要求します。外側の標準TransactionNotificationはsolverにしか適用されないので、makerも専用validatorで確認します。
- 既存のAccountPublicKey/AccountAddress、AddressInteraction、BalanceTransfer通知を適切な順で発行し、住所制限、mosaic受取制限、global/address restriction、transferability、残高検査を通します。
- Balance cacheへ直接書き込んで既存制限を省かないでください。
- makerの操作制限検証のために偽のTransactionNotificationを発行してはいけません。HashCache observer等まで誤って発火します。
- Cancelは通常の外側認可と期限・未消費を要求し、対象資産の現在残高や存在を取消条件にしません。

2脚目または標準feeの処理が失敗した場合に、1脚目・公開鍵登録・consumption・receiptが残らないことを検証してください。単純な「前段で全部チェックしたから安全」という推測でundo試験を省略してはいけません。

5. 消費状態を合意状態にする

keyはorderId32B。valueはstatus1B＋reserved7B＋makerAddress24B＋validUntil8B＋consumedHeight8B＋outerTxHash32B＝80B。statusはFILLED/CANCELLEDだけを保存します。未登録はOPEN、期限切れは注文データとblockTimeからEXPIREDを導出します。保存済み終端状態は期限後も保持します。

新しいMerkle対応sub-cacheを登録し、cache serializer、patricia/root、永続化、commit/rollback、再起動を実装してください。ローカルJSON、メモリset、REST照会結果で置き換えないでください。PoCではレコードをpruneしません。両操作は同じcacheを使い、同block内の先行deltaを後行validatorが見られるようにします。

新cacheはstateHash構成に影響します。nemgenにも同じプラグインを導入してfresh genesisを生成し、3ノードの設定とcache構成を一致させてください。既存mainnetのgenesisやstate snapshotに後付けして成功扱いにしないでください。mainnet移行の実装は範囲外です。

6. 段階的に進める

M0: ソースと実行環境の確認

baseline commit、適用AGENTS、build/test手順、pluginの登録例、既存cacheのserializerとundo、schema generator、nemgenを確認します。設計書に書かれた提案APIが実在すると仮定せず、実ソースの対応表を作ってください。隔離private-net用設定、chain seed、通貨mosaic、private type/cache割当を記録します。

M1〜M3: codec、plugin、状態

固定fixtureを最初に通し、その後C++取引型、validators、既存transfer経路、cacheを実装します。新型だけを追加してmaker制限を飛ばす近道は不可です。コア変更が必要なら、理由・最小差分・検証範囲を分離してください。プラグインだけで済んだという見かけを優先しないでください。

M4〜M5: 実ネットワークと分岐

3ノード全てを実物Catapultと新プラグインで起動します。非ゼロfeeで1件ずつ正当に決済・取消し、全ノードのblock hash、stateHash、receiptsHash、各口座残高、消費状態を比較します。その後、取消競争、重複、無効注文、非finalizedの分岐とrollback、再起動を試します。任意の深いfinalized rollbackを通常プロトコルとして許す変更はしないでください。

ローカル候補の失敗は外側1件をundoして除外します。一方、無効なSettlementが受信blockに入っている場合はblock全体を拒否します。この2つを別試験にします。

M6: 再現性と引渡し

設計書のIS-01〜IS-20を1行ずつ報告し、実行済み・失敗・未実行・環境blockを区別します。fixtureだけ通った状態をPoC完成と呼ばないでください。

7. 必須の試験と受入判定

少なくとも、署名field改変／wrong seed、expiry境界、alias拒否、maker/solver multisig、双方の操作制限、住所とmosaic制限、第2脚失敗、非ゼロfee、get/giveがXYMの両方向、同block内の二重消費、cancel/settleの両順、他者取消拒否、UT undo、無効受信block、reorg、再起動、genesis/root一致を実行してください。

各試験について、対象commit、実行コマンド、node/plugin設定、exit code、raw log、tx/block hash、前後残高、consumption record、root/hashの比較結果を保存します。syntheticなテスト鍵だけを使い、秘密鍵は成果物ログへ出さないでください。

最終報告では次を分けます。

- 実装済みの機能と実際に通ったC++ unit/integration tests。
- 3ノードで実行できたケースと再現手順。
- NOT RUN、失敗、環境依存、まだ証明していない性質。
- source差分と設計からの変更理由。
- mainnetには導入できない現時点の範囲。

受入は、C++プラグイン＋3ノードの実行証跡＋失敗・rollback試験が揃った時点です。ネットワークや依存関係で試験ができない場合も実装・静的確認・実行可能な試験を進め、残るblockを正確に示してください。未実行をPASSへ書き換えてはいけません。